Análisis comparativo de Bubble Sort y Selection Sort en el ATmega328P

Flores Mondragón José Adrián, Tapia Ortega Citlalli Yael
Universidad Nacional Autónoma de México

Facultad de Estudios Superiores Cuautitlán
16 de Marzo de 2026

Resumen

En este trabajo se implementaron y compararon dos algoritmos de ordenamiento, Bubble Sort y Selection Sort, en el microcontrolador ATmega328P utilizando lenguaje ensamblador AVR. El programa genera un arreglo de 100 números pseudoaleatorios en el rango de 0 a 100, lo almacena en memoria SRAM y crea dos copias del arreglo original para que cada algoritmo trabaje con la misma entrada, mide el tiempo de ejecución de cada uno haciendo uso del Timer1 del microcontrolador. Se comparan los resultados experimentales con la complejidad teórica expresada en notación Big-O. Con los datos obtenidos se observó que ambos algoritmos ordenaron correctamente el arreglo, pero en esta implementación Bubble Sort resultó más rápido que Selection Sort, lo que muestra que el rendimiento real no depende solo de la complejidad teórica, sino también de la forma en que cada rutina fue programada y del costo de acceso a memoria en la arquitectura AVR.

I. Introducción

Los algoritmos de ordenamiento son fundamentales en ciencias de la computación [5] porque permiten organizar datos para facilitar su búsqueda, clasificación y procesamiento. En el caso del microcontrolador ATmega328P, el rendimiento de un programa depende del número de instrucciones ejecutadas, de los accesos a memoria SRAM y del uso de registros y punteros.[1], [2]. Por esa razón se compararon dos algoritmos: Bubble Sort y Selection Sort.

El objetivo fue medir el tiempo en que cada algoritmo ordena el mismo arreglo de 100 números y comparar los resultados. Para hacer una comparación justa y observar cuál de los dos métodos fue más conveniente primero se generó un arreglo pseudoaleatorio, después se copió a dos tablas independientes y se ejecutó un algoritmo en cada copia.

II. Marco teórico

A. Arquitectura AVR y memoria SRAM

El ATmega328P es un microcontrolador de 8 bits basado en arquitectura AVR.[1] En esta práctica, la SRAM se utilizó para guardar el arreglo original, los arreglos de salida de cada algoritmo, el estado del generador pseudoaleatorio y las variables donde se almacenaron los tiempos medidos. En ensamblador AVR, el acceso a arreglos se realiza con punteros indirectos, usando los registros X, Y y Z.[2]

B. Generación de números pseudoaleatorios

Para poblar el arreglo original se usó una rutina basada en un estado de 8 bits. El procedimiento consiste en leer el estado

actual, desplazarlo a la derecha, revisar el bit menos significativo y aplicar una operación XOR. Después, el valor se reduce entre 0 - 100 mediante restas de 101.

C. Notación Big-O

Se utiliza para describir el crecimiento del costo de un algoritmo conforme aumenta el tamaño de la entrada.[5] En este trabajo se analizaron dos algoritmos con complejidad temporal cuadrática, es decir, de orden $O(n^2)$. Esto significa que, cuando el número de datos aumenta, el tiempo de ejecución crece de manera proporcional al cuadrado del tamaño de entrada.

D. Bubble Sort

Compara elementos vecinos e intercambia sus posiciones cuando están en el orden incorrecto. Este proceso se repite hasta que el arreglo queda completamente ordenado. Es un algoritmo sencillo, pero puede requerir una gran cantidad de comparaciones e intercambios.[3]

E. Selection Sort

Recorre el arreglo buscando el valor mínimo de la parte no ordenada y lo coloca en la posición correcta. Aunque presenta complejidad $O(n^2)$, realiza menos intercambios que Bubble Sort. Sin embargo, su eficiencia real depende de cómo se implemente sobre la arquitectura utilizada.[4]

Algoritmo	Mejor caso	Peor caso
Bubble Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$

III. Desarrollo e implementación

El programa se desarrolló en lenguaje ensamblador AVR. Se definieron dos constantes principales: N = 100, que representa el tamaño del arreglo, y MODVAL = 101, usada para limitar los números pseudoaleatorios al intervalo de 0 a 100.

En la memoria SRAM se reservaron tres arreglos de 100 bytes: uno para los datos originales desordenados y dos para los resultados de cada algoritmo. Además, se incluyó una variable para el estado del generador pseudoaleatorio y dos variables de dos bytes para almacenar el tiempo medido de cada rutina.

La rutina principal inicializa el stack, configura el Timer1 en modo normal y establece un valor inicial para la generación de datos. Después llena el arreglo original con 100 números pseudoaleatorios y lo copia a dos tablas independientes. Una de esas tablas se ordena con Bubble Sort y la otra con Selection Sort. Antes de ejecutar cada algoritmo, el temporizador se reinicia, y al finalizar se guardan los ciclos de reloj medidos en memoria.

Esta organización permitió que ambos métodos trabajaran con la misma entrada, así la comparación entre ellos fueválida.

Pseudocódigo del funcionamiento general

Arreglo original de 100 elementos, Timer1, dos arreglos auxiliares
Resultado: dos arreglos ordenados y tiempos de ejecución almacenados

1. **Inicio**
2. Inicializar el stack del microcontrolador
3. Configurar el Timer1 en modo normal
4. Inicializar el estado del generador pseudoaleatorio

5. Llenar el arreglo original con 100 números pseudoaleatorios
6. Copiar el arreglo original al arreglo destinado a Bubble Sort
7. Reiniciar el Timer1
8. Ejecutar Bubble Sort
9. Almacenar el tiempo medido del algoritmo 1
10. Copiar el arreglo original al arreglo destinado a Selection Sort
11. Reiniciar el Timer1
12. Ejecutar Selection Sort
13. Almacenar el tiempo medido del algoritmo 2
14. Mantener el programa en espera
15. **Fin**

Ordenamiento por Bubble Sort

Arreglo desordenado de 100 elementos
Arreglo ordenado en forma ascendente

1. **Inicio**
2. Establecer el número de pasadas del algoritmo
3. Recorrer el arreglo comparando elementos vecinos
4. Si el elemento actual es mayor que el siguiente, intercambiar sus posiciones
5. Continuar el recorrido hasta completar una pasada
6. Reducir el número de comparaciones restantes
7. Repetir el proceso hasta ordenar completamente el arreglo
8. **Fin**

Generador pseudoaleatorio

Estado actual del generador
Número pseudoaleatorio en el intervalo de 0 a 100

1. **Inicio**
2. Leer el estado actual del generador
3. Obtener el bit menos significativo
4. Desplazar el valor una posición a la derecha
5. Si el bit menos significativo es 1, aplicar una operación XOR
6. Actualizar el estado del generador
7. Reducir el valor hasta que se encuentre en el intervalo de 0 a 100
8. Regresar el número generado
9. **Fin**

Ordenamiento por Selection Sort

1. **Inicio**
2. Seleccionar una posición inicial del arreglo
3. Suponer que dicha posición contiene el valor mínimo
4. Recorrer la parte no ordenada del arreglo
5. Comparar cada elemento con el mínimo actual
6. Si se encuentra un valor menor, actualizar la posición del mínimo
7. Al finalizar el recorrido, intercambiar el valor mínimo con la posición inicial
8. Avanzar a la siguiente posición del arreglo
9. Repetir el proceso hasta ordenar todos los elementos
10. **Fin**

IV. Metodología experimental

Para evaluar el desempeño de los algoritmos se utilizó la lectura de memoria del sistema. La prueba consistió en ejecutar el programa completo, verificar el contenido de los arreglos finales y leer las variables donde se almacenaron los ticks medidos por el Timer1.

Con base en los resultados visibles en memoria, fue posible confirmar que ambos algoritmos produjeron un arreglo ordenado de manera ascendente. También se identificaron los bytes correspondientes a las variables de tiempo del algoritmo 1 y del algoritmo 2.

V. Resultados y análisis

Los resultados obtenidos muestran que ambos algoritmos ordenaron correctamente el arreglo de 100 elementos. Sin embargo, los tiempos medidos fueron diferentes: Bubble Sort registró 1126 cuentas del temporizador, mientras que Selection Sort registró 1362 cuentas, por lo que Bubble Sort fue más rápido en esta implementación. La diferencia fue de 236 cuentas del temporizador.

Los tres experimentos mostraron exactamente los mismos valores, esto indica que el sistema fue estable y repetible bajo las mismas condiciones de ejecución. No se modificaron variables como el tamaño del arreglo, el valor inicial del generador ni la configuración del temporizador. Por ello, los resultados obtenidos corresponden a repeticiones del mismo caso de prueba.

Aunque teóricamente ambos algoritmos pertenecen a la clase $O(n^2)$, en la práctica su desempeño no fue idéntico.[3]-[5]Esto demuestra que la complejidad describe el crecimiento general del algoritmo, pero no el costo real de cada instrucción. En este caso, la organización interna de Bubble Sort resultó más eficiente que la de Selection Sort.

Experimento	**Bubble Sort (cuentas)**	**Selection Sort (cuentas)**	**Bubble Sort (ms)**	**Selection Sort (ms)**	**Más rápido**
1	1126	1362	4.504	5.448	Bubble Sort
2	1126	1362	4.504	5.448	Bubble Sort
3	1126	1362	4.504	5.448	Bubble Sort

Criterio	**Bubble Sort**	**Selection Sort**
Complejidad teórica	$O(n^2)$	$O(n^2)$
Comparaciones teóricas para n = 100	4950	4950
Cuentas experimentales	1126	1362
Tiempo estimado (ms)	4.504	5.448
Resultado práctico	Más rápido	Más lento

VI. Conclusión

En este trabajo se implementaron y compararon los algoritmos de ordenamiento, Bubble Sort y Selection Sort. El sistema fue capaz de generar un arreglo de 100 números pseudoaleatorios, almacenarlo en SRAM y ordenar dos copias idénticas del mismo, lo que permitió realizar una comparación justa entre ambos métodos.

Los resultados experimentales confirmaron que los dos algoritmos funcionaron correctamente, ya que ambos ordenaron de manera el arreglo generado. Sin embargo, al comparar los tiempos medidos se observó que Bubble Sort registró 1126 cuentas del temporizador, mientras que Selection Sort registró 1362, por lo que Bubble Sort resultó más rápido en esta implementación. Además, las tres ejecuciones realizadas produjeron los mismos valores, lo que indica que el sistema presentó un comportamiento estable, consistente y repetible bajo las mismas condiciones de prueba.

Aunque desde el punto de vista teórico ambos algoritmos pertenecen a la clase de complejidad $O(n^2)$, la práctica demostró que el rendimiento real no depende de la teoría, sino también de la organización del código, el manejo de punteros, la cantidad de instrucciones ejecutadas y el costo de acceso a memoria dentro de la arquitectura AVR.

Referencias

1] Microchip Technology Inc., “ATmega328P Datasheet,” *Microchip Technology*. [En línea]. Consultado: 16-mar-2026.

[2] Microchip Technology Inc., “AVR® Instruction Set Manual,” *Microchip Technology*. [En línea]. Consultado: 16-mar-2026.

[3] GeeksforGeeks, “Bubble Sort,” *GeeksforGeeks*. [En línea]. Consultado: 16-mar-2026.

[4] GeeksforGeeks, “Selection Sort,” *GeeksforGeeks*. [En línea]. Consultado: 16-mar-2026.

[5] freeCodeCamp, “Explicación de la notación Big O con ejemplo,” *freeCodeCamp en Español*. [En línea]. Consultado: 16-mar-2026.